

Feature Scaling

Standardization vs Normalization A Complete Reference Guide for ML Practitioners

Amol Jagtap | amoljagtap3001@gmail.com

Field	Detail
Topic	Feature Scaling — Normalization and Standardization
Pre-requisite	Feature Engineering Data Preprocessing Basics
Libraries	scikit-learn, pandas, numpy, matplotlib, seaborn
Key Classes	StandardScaler, MinMaxScaler, RobustScaler, MaxAbsScaler
Key Takeaway	Scale your features correctly or your model will silently underperform

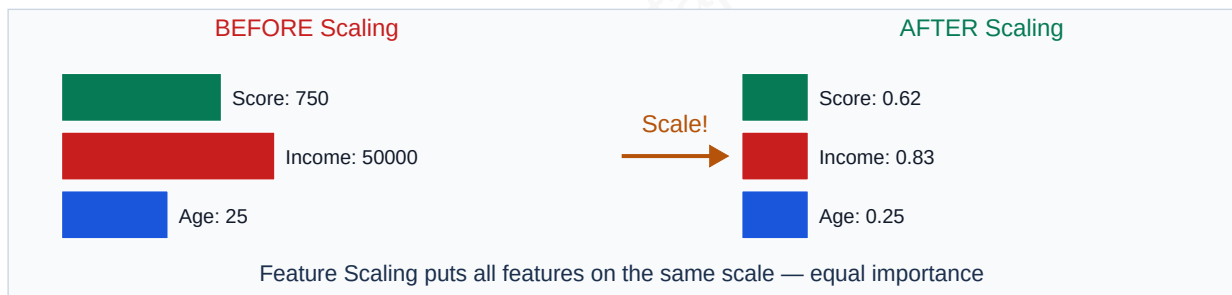


Figure 1 — Without scaling, large-value features dominate; scaling puts all features on equal footing

Table of Contents

1. What is Feature Scaling?
2. Why Feature Scaling Matters
3. Min-Max Normalization
4. Standardization (Z-Score Scaling)
5. Min-Max vs Standardization — Full Comparison
6. RobustScaler — Handling Outliers
7. MaxAbsScaler and Other Scalers
8. Which Scaler to Use — Algorithm Guide
9. How to Apply Scaling Correctly — Data Leakage
10. sklearn Implementation — Full Code
11. Scaling in a Pipeline
12. Quick Reference Cheat Sheet

Amol Jagtap | amoljagtap3001@gmail.com

1. What is Feature Scaling?

Feature Scaling is a preprocessing technique that changes the range or distribution of numeric features so they all fall on a comparable scale. It does NOT change the shape or ordering of the data — just the magnitude.

Without Scaling	With Scaling
Age: 18 – 70	Age: 0.0 – 1.0 (or -2 to +2)
Income: 20,000 – 5,00,000	Income: 0.0 – 1.0
Credit Score: 300 – 850	Credit Score: 0.0 – 1.0
Income dominates — model is biased	All features contribute equally

■ Feature scaling does NOT create new features. It only transforms existing numeric columns. Categorical columns must be encoded first, then scaled if needed.

2. Why Feature Scaling Matters

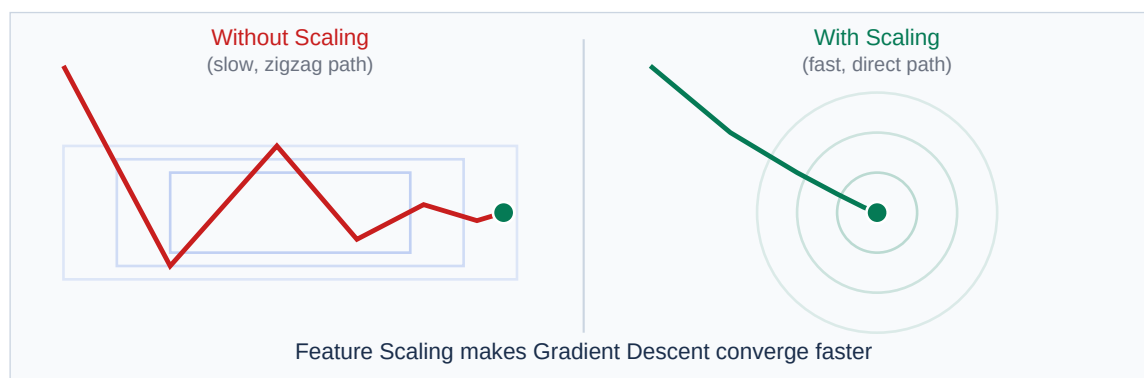


Figure 2 — Without scaling, gradient descent zigzags slowly; with scaling it converges in a direct path

2.1 The Euclidean Distance Problem

Many ML algorithms compute distances between data points — KNN, SVM, K-Means, PCA. Euclidean distance between two points depends heavily on the scale of each feature. A feature with range 0–100,000 will completely overwhelm one with range 0–1.

$$\text{Distance} = \sqrt{(\text{Age2} - \text{Age1})^2 + (\text{Income2} - \text{Income1})^2}$$

Example: Age difference of 10 years gives $(10)^2 = 100$. Income difference of Rs 10,000 gives $(10000)^2 = 100,000,000$. The model effectively ignores Age completely!

2.2 Gradient Descent Convergence

In gradient descent (used by linear regression, logistic regression, neural networks), the loss surface becomes elongated when features have different scales. The optimizer zigzags and takes many more steps to reach the minimum. Scaling makes the loss surface circular, allowing direct convergence.

2.3 Regularisation Fairness

Lasso (L1) and Ridge (L2) regularisation penalise large coefficients. If Income has coefficients in the range of 0.001 and Age has 5.0, regularisation punishes Age more — not because it is less important, but because it has a different scale. Scaling fixes this.

Algorithm	Needs Scaling?	Why
K-Nearest Neighbors	YES	Distance-based — dominated by large features
Support Vector Machine	YES	Maximises margin — scale-sensitive
K-Means Clustering	YES	Uses Euclidean distance for centroids
PCA	YES	Maximises variance — large-scale features dominate
Linear Regression	YES	Regularisation fairness + gradient descent
Logistic Regression	YES	Regularisation + gradient descent
Neural Networks	YES	Gradient descent + weight initialisation

Algorithm	Needs Scaling?	Why
Decision Tree	NO	Splits on thresholds — scale independent
Random Forest	NO	Ensemble of trees — scale independent
XGBoost / LightGBM	NO	Tree-based — scale independent
Naive Bayes	NO	Probability-based — not distance or gradient

Amol Jagtap | amoljagtap3001@gmail.com

3. Min-Max Normalization

Min-Max Normalization (also called Min-Max Scaling) compresses all values into a fixed range — usually 0 to 1. The minimum value becomes 0, the maximum becomes 1, and all other values are proportionally placed between them.

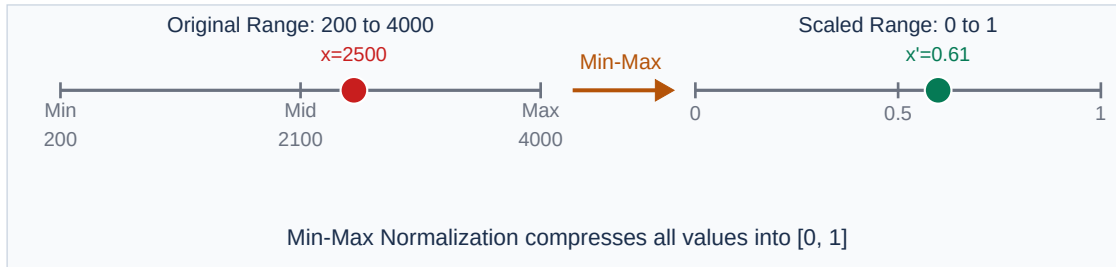


Figure 3 — Min-Max maps the original range onto [0, 1]

Formula

$$x' = (x - x_{\min}) / (x_{\max} - x_{\min})$$

Worked Example

Original Value (x)	x_min	x_max	Scaled Value (x')
200	200	4000	$(200-200)/(4000-200) = 0.000$
2500	200	4000	$(2500-200)/(4000-200) = 0.605$
4000	200	4000	$(4000-200)/(4000-200) = 1.000$

sklearn Implementation

```
from sklearn.preprocessing import MinMaxScaler import numpy as np # Sample data X =  
np.array([[200], [2500], [1000], [3500], [4000]]) # Create scaler and fit on TRAINING  
data only scaler = MinMaxScaler() X_train_scaled = scaler.fit_transform(X_train) #  
Transform test data using SAME scaler X_test_scaled = scaler.transform(X_test) # Check  
results print('Min:', scaler.data_min_) print('Max:', scaler.data_max_) print('Scaled  
sample:', X_train_scaled[:5]) # Custom range [a, b] instead of [0, 1] scaler_custom =  
MinMaxScaler(feature_range=(-1, 1)) X_scaled_custom =  
scaler_custom.fit_transform(X_train)
```

Advantage	Disadvantage
Output is always in [0, 1]	Very sensitive to outliers
Preserves the original distribution	Outlier shifts min/max dramatically
Good for neural networks and image data	Does not centre data at 0
Intuitive — easy to interpret	Breaks if test data has new min/max

■ If your test data has a value outside the training min/max range, MinMaxScaler will produce values outside [0,1]. Always check for new extremes at inference time.

4. Standardization (Z-Score Scaling)

Standardization rescales features so they have a mean of 0 and a standard deviation of 1. Unlike Min-Max, it does NOT bound the output to a fixed range — values can go beyond -3 or +3 (especially for outliers). It is the most widely used scaling method in ML.

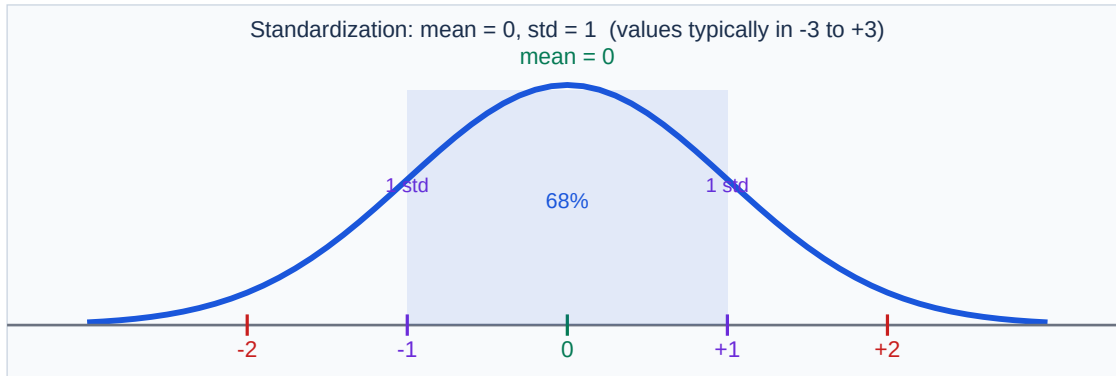


Figure 4 — After standardization: mean = 0, std = 1; 68% of values lie within ± 1 standard deviation

Formula

$$z = (x - \text{mean}) / \text{std}$$

Worked Example

Original (x)	mean	std	Z-Score $z = (x - \text{mean}) / \text{std}$
30	50	20	$(30 - 50) / 20 = -1.0$
50	50	20	$(50 - 50) / 20 = 0.0$ (mean)
70	50	20	$(70 - 50) / 20 = +1.0$
90	50	20	$(90 - 50) / 20 = +2.0$
10	50	20	$(10 - 50) / 20 = -2.0$

sklearn Implementation

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler() # Fit on training data ONLY
X_train_scaled = scaler.fit_transform(X_train) # Apply SAME scaler to test
X_test_scaled = scaler.transform(X_test) # Inspect what the scaler learned
print('Mean per feature:', scaler.mean_)
print('Std per feature:', scaler.scale_)
# Reverse the scaling (go back to original values)
X_original = scaler.inverse_transform(X_train_scaled)
```

Advantage	Disadvantage
Most widely used — works for most models	Output range is unbounded
Not sensitive to outliers (uses std, not range)	Values not in [0,1] — some models prefer bounded
Works well with regularisation (L1, L2)	Assumes roughly normal distribution

Advantage	Disadvantage
Gradient descent converges faster	Slightly less interpretable than Min-Max

Amol Jagtap | amoljagtap3001@gmail.com

5. Min-Max vs Standardization — Full Comparison

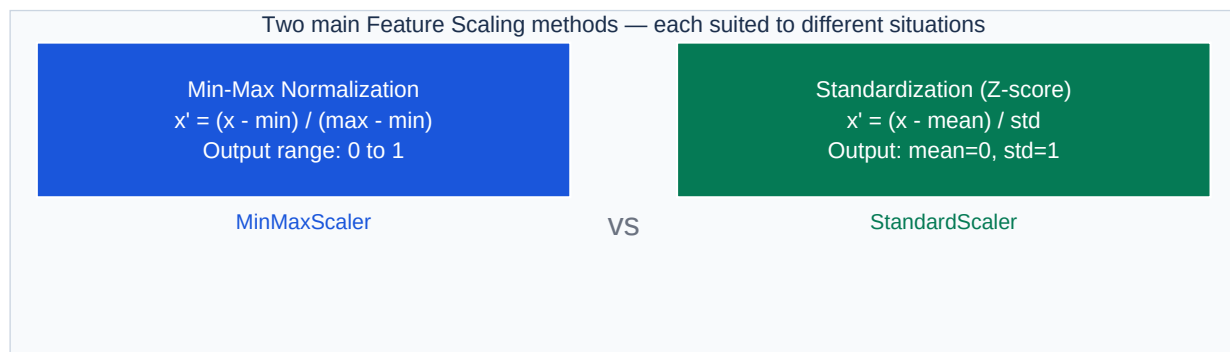


Figure 5 — Side-by-side summary of both scaling methods

Property	Min-Max Normalization	Standardization
sklearn class	MinMaxScaler()	StandardScaler()
Formula	$(x - \min) / (\max - \min)$	$(x - \text{mean}) / \text{std}$
Output range	Fixed: [0, 1]	Unbounded (~-3 to +3)
Mean of output	Not necessarily 0	Always 0
Std of output	Not necessarily 1	Always 1
Effect of outliers	Very high — distorts range	Lower — std absorbs them
Distribution preserved	Yes — same shape	Yes — same shape
Makes data normal?	No	No
Best for	Neural nets, image pixels, KNN	Most ML models, PCA, SVM, Lasso
When to prefer	When you know bounds are fixed	When distribution is approximately normal

■ Neither Normalization nor Standardization makes your data normally distributed. To change the distribution shape, use a log transform or Box-Cox (see Feature Engineering notes).

6. RobustScaler — Handling Outliers

When your dataset has significant outliers, MinMaxScaler is badly distorted (outlier becomes the new max) and StandardScaler is partially affected (outlier inflates the std). RobustScaler solves this by using the median and IQR instead of mean and std — statistics that are not affected by extreme values.

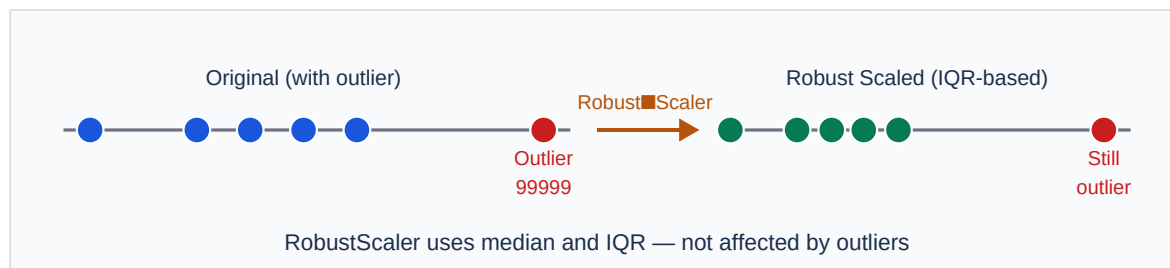


Figure 6 — RobustScaler uses IQR; outliers are reduced but not eliminated

Formula

$$x' = (x - \text{median}) / \text{IQR where } \text{IQR} = Q3 - Q1$$

sklearn Implementation

```
from sklearn.preprocessing import RobustScaler scaler = RobustScaler(
quantile_range=(25.0, 75.0) # default IQR = Q1 to Q3 ) X_train_scaled =
scaler.fit_transform(X_train) X_test_scaled = scaler.transform(X_test) print('Median:',
scaler.center_) print('IQR :', scaler.scale_)
```

Property	RobustScaler
Centre	Subtracts the median (not mean)
Scale	Divides by IQR = Q3 - Q1 (not std)
Output range	Unbounded (similar to standardization)
Outlier effect	Very low — median and IQR are robust statistics
Best for	Data with significant outliers that should be kept
sklearn class	RobustScaler()

✓ **Tip: Use RobustScaler when you have many outliers that are real data points (e.g. income distributions, fraud amounts, medical measurements). Use StandardScaler for clean, roughly normal data.**

7. MaxAbsScaler and Other Scalers

MaxAbsScaler

Scales each feature by its maximum absolute value. The result is in the range $[-1, 1]$. It does NOT shift the data (no mean subtraction), making it ideal for sparse data (many zeros) such as text TF-IDF matrices.

$$x' = x / \max(|x|)$$

```
from sklearn.preprocessing import MaxAbsScaler scaler = MaxAbsScaler() X_scaled =  
scaler.fit_transform(X_train) # Result: values in [-1, 1], sparse structure preserved
```

Normalizer (L2 Row Normalisation)

Normalizer scales each individual sample (row) to have unit norm (length = 1). Unlike all other scalers which work column-wise, Normalizer works row-wise. Used in text classification and cosine similarity tasks.

```
from sklearn.preprocessing import Normalizer scaler = Normalizer(norm='l2') # l1, l2,  
or max X_scaled = scaler.fit_transform(X_train) # Each ROW has L2 norm = 1
```

Scaler	Formula	Output Range	Best Use Case
MinMaxScaler	$(x - \min) / (\max - \min)$	$[0, 1]$	Neural nets, image data, KNN
StandardScaler	$(x - \text{mean}) / \text{std}$	$\sim [-3, 3]$	Most algorithms, PCA, SVM, Lasso
RobustScaler	$(x - \text{median}) / \text{IQR}$	Unbounded	Data with outliers
MaxAbsScaler	$x / \max(x)$	$[-1, 1]$	Sparse data, text (TF-IDF)
Normalizer (L2)	$x / \ x\ $	Unit norm	Text, cosine similarity

8. Which Scaler to Use — Algorithm Guide

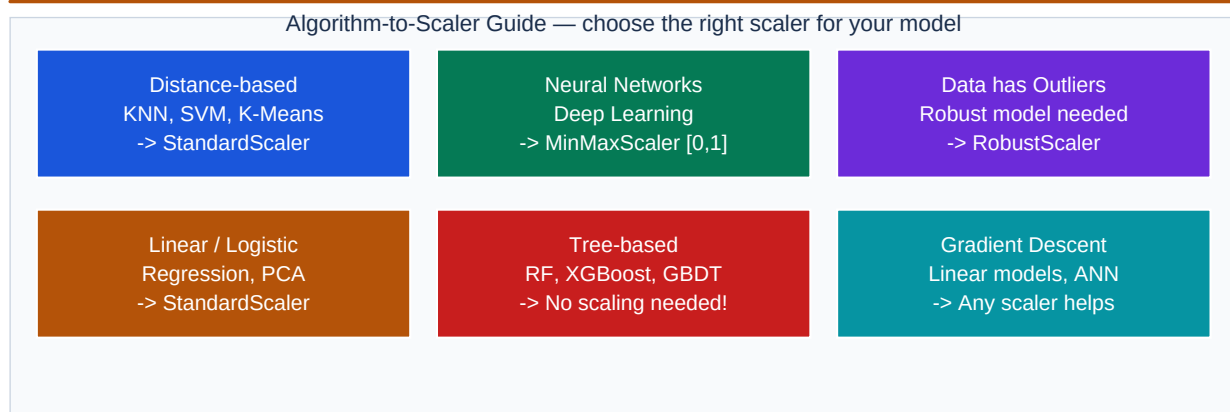


Figure 7 — Quick algorithm-to-scaler decision guide

Your Situation	Recommended Scaler	Reason
General ML, no strong preference	StandardScaler	Works well for most algorithms
Neural network / deep learning	MinMaxScaler [0,1]	Sigmoid/tanh activations prefer [0,1]
KNN, SVM, K-Means	StandardScaler	Distance-based, needs zero-centred
PCA, LDA	StandardScaler	Variance must be comparable
Data has many outliers	RobustScaler	Median/IQR not affected by extremes
Sparse text data (TF-IDF, BoW)	MaxAbsScaler	Does not destroy sparsity
Decision Tree, RF, XGBoost	No scaling needed	Tree splits are scale-invariant
Lasso / Ridge regression	StandardScaler	Regularisation penalises equally
Logistic regression (binary)	StandardScaler	Gradient descent + regularisation
Image pixel data (0-255)	MinMaxScaler or /255	Pixels already bounded

9. How to Apply Scaling Correctly — Avoiding Data Leakage

Data leakage is one of the most common and dangerous mistakes in ML. It occurs when information from the test set accidentally influences the training process — producing over-optimistic evaluation scores that do not hold in the real world. Incorrect scaling is one of the primary causes of leakage.

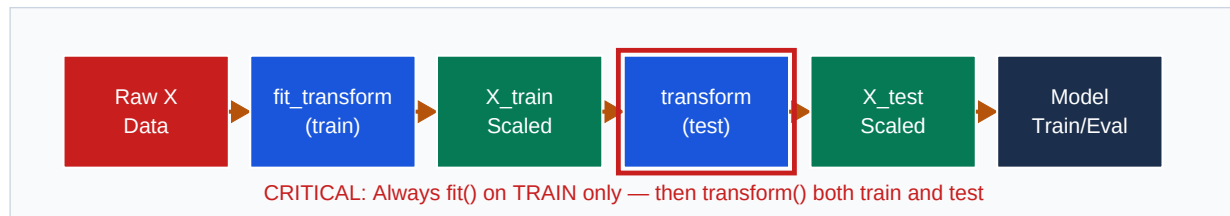


Figure 8 — CRITICAL: `fit()` ONLY on training data; `transform()` on both. Never call `fit_transform()` on test data.

The Golden Rule

Step	Training Set	Test Set
Fit	<code>scaler.fit(X_train)</code> YES — learn mean, std, min, max from here	NEVER — do not fit on test
Transform	<code>scaler.transform(X_train)</code> Apply learned parameters	<code>scaler.transform(X_test)</code> Apply SAME parameters
Combined	<code>scaler.fit_transform(X_train)</code> — shortcut	<code>scaler.transform(X_test)</code> — NOT <code>fit_transform</code>

Wrong Way vs Right Way

```
# WRONG – leaks test statistics into the scaler scaler.fit(X) # fitted on ALL data
X_train_s = scaler.transform(X_train) # biased X_test_s = scaler.transform(X_test) #
mean/std already includes test # CORRECT – only learn from training data from
sklearn.model_selection import train_test_split X_train, X_test, y_train, y_test =
train_test_split( X, y, test_size=0.2, random_state=42) scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train) # fit + transform train X_test_s =
scaler.transform(X_test) # transform test ONLY
```

■ Calling `scaler.fit()` or `scaler.fit_transform()` on the test set is data leakage. The model will see the test distribution during training, producing unrealistically high scores that fail in production.

10. sklearn Implementation — Full Code

[illegible]

11. Scaling in a Pipeline

The cleanest and safest way to apply feature scaling in a real project is inside a sklearn Pipeline. A Pipeline chains preprocessing and modelling steps together, automatically applying `fit()` on training data and `transform()` on test data — eliminating any risk of data leakage.

[illegible]

✓ Tip: Always use a Pipeline in production. It ensures the scaler is saved together with the model, so when you deploy and get new data, scaling is applied identically.

```
import joblib # Save the full pipeline (scaler + model together) joblib.dump(pipe_full, '
model_pipeline.pkl') # Load and predict on new data – scaling happens automatically
loaded = joblib.load('model_pipeline.pkl') predictions = loaded.predict(X_new) # X_new
is raw, unscaled!
```

12. Quick Reference Cheat Sheet

Task	Code
Import StandardScaler	<code>from sklearn.preprocessing import StandardScaler</code>
Import MinMaxScaler	<code>from sklearn.preprocessing import MinMaxScaler</code>
Import RobustScaler	<code>from sklearn.preprocessing import RobustScaler</code>
Import MaxAbsScaler	<code>from sklearn.preprocessing import MaxAbsScaler</code>
Fit on train only	<code>scaler.fit(X_train)</code>
Fit + transform train	<code>X_tr = scaler.fit_transform(X_train)</code>
Transform test (same scaler)	<code>X_te = scaler.transform(X_test)</code>
NEVER do this	<code># scaler.fit_transform(X_test) <- LEAKAGE</code>
Get mean (StandardScaler)	<code>scaler.mean_</code>
Get std (StandardScaler)	<code>scaler.scale_</code>
Get min (MinMaxScaler)	<code>scaler.data_min_</code>
Get max (MinMaxScaler)	<code>scaler.data_max_</code>
Reverse scaling	<code>scaler.inverse_transform(X_scaled)</code>
Min-Max formula	$x' = (x - x_{\min}) / (x_{\max} - x_{\min})$
Standardization formula	$z = (x - \text{mean}) / \text{std}$
RobustScaler formula	$x' = (x - \text{median}) / \text{IQR}$
Build scaling pipeline	<code>Pipeline([('sc', StandardScaler()), ('m', model)])</code>
Cross-validate with pipeline	<code>cross_val_score(pipe, X, y, cv=5)</code>
Save pipeline	<code>joblib.dump(pipe, 'pipeline.pkl')</code>
Load pipeline	<code>joblib.load('pipeline.pkl')</code>
Use for most models	<code>StandardScaler()</code>
Use for neural networks	<code>MinMaxScaler(feature_range=(0,1))</code>
Use with outliers	<code>RobustScaler()</code>
Use for sparse data	<code>MaxAbsScaler()</code>
Use for tree models	<code># No scaling needed</code>
Check skewness before scaling	<code>df['col'].skew() # 0 = symmetric</code>
Visualise distribution	<code>df['col'].hist(bins=30)</code>

Task	Code
Compare all scalers	for sc in [Standard, MinMax, Robust]: sc.fit_transform(X)

Amol Jagtap | amoljagtap3001@gmail.com

Amol Jagtap | amoljagtap3001@gmail.com